Operating Systems

Dr Hamela K
Department of Computer Science
GFGC, Malur

Unit-I

Introduction: Definition, Types of Operating Systems, Functions of Operating System, services, system components System call. Process Management: Process Concept, Process Scheduling, Inter process communication, CPU Scheduling Criteria, Scheduling algorithm, Multiple Processor Scheduling, Real time Scheduling, Algorithm evolution

What is OS?

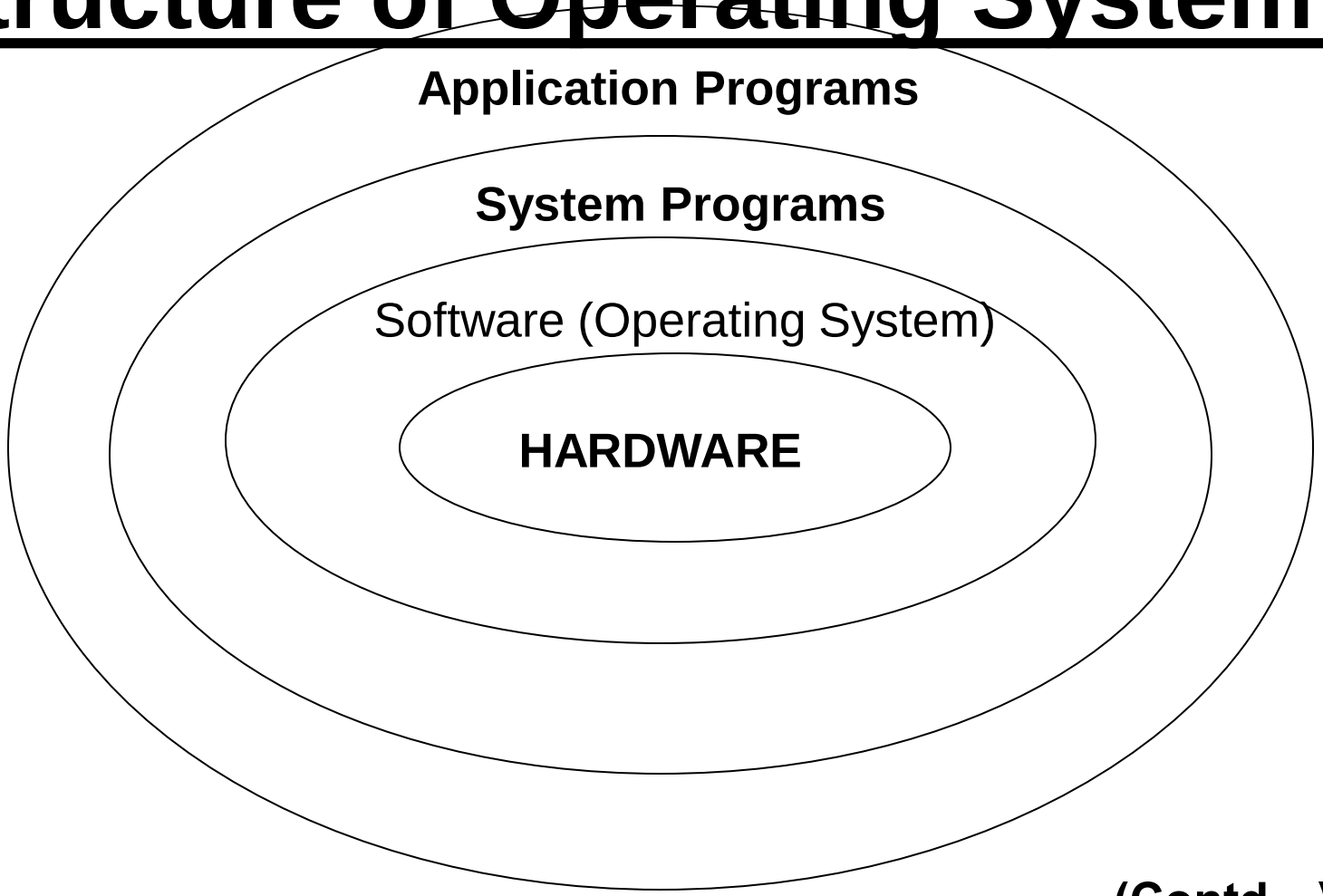
- Operating System is a software, which makes a computer to actually work.
- It is the software the enables all the programs we use.
- The OS organizes and controls the hardware.
- OS acts as an interface between the application programs and the machine hardware.
- Examples: Windows, Linux, Unix and Mac OS, etc.,

What OS does?

An operating system performs basic tasks such as,

- controlling and allocating memory,
- prioritizing system requests,
- controlling input and output devices,
- facilitating networking and
- managing file systems.

Structure of Operating System:



(Contd…)

Structure of Operating System (Contd…):

- The structure of OS consists of 4 layers:
    1. **Hardware**

       Hardware consists of CPU, Main memory, I/O Devices, etc,

    2. **Software (Operating System)**

       Software includes process management routines, memory management routines, I/O control routines, file management routines.

(Contd…)

Structure of Operating System (Contd…):

3. **System programs**

   This layer consists of compilers, Assemblers, linker etc.

4. **Application programs**

   This is dependent on users need. Ex. Railway reservation system, Bank database management etc.,

Evolution of OS:

- The evolution of operating systems went through seven *major phases.*

- Six of them significantly changed the ways in which users accessed computers through the open shop, batch processing, multiprogramming, timesharing, personal computing, and distributed systems.

- In the seventh phase the foundations of concurrent programming were developed and demonstrated in model operating systems.

(Contd…)

Evolution of OS (contd..):

Major Phases	Technical Innovations	Operating Systems
Open Shop	**The idea of OS**	**IBM 701 open shop (1954)**
Batch Processing	**Tape batching,** **First-in, first-out scheduling.**	**BKS system (1961)**
Multi-programming	**Processor multiplexing, Indivisible operations, Demand paging, Input/output spooling, Priority scheduling, Remote job entry**	**Atlas supervisor (1961),** **Exec II system (1966)**

(Contd…)

Evolution of OS (contd..):

Timesharing	**Simultaneous user interaction,** **On-line file systems**	**Multics file system (1965),** **Unix (1974)**
Concurrent Programming	**Hierarchical systems, Extensible kernels, Parallel programming concepts, Secure parallel languages**	**RC 4000 system (1969),** **13 Venus system (1972),** **14 Boss 2 system (1975).**
Personal Computing	**Graphic user interfaces**	**OS 6 (1972)** **Pilot system (1980)**
Distributed Systems	**Remote servers**	**WFS file server (1979) Unix United RPC (1982)** **24 Amoeba system (1990)**

Operating Systems functions:

- The main functions of operating systems are:

1. Program creation
2. Program execution
3. Input/Output operations
4. Error detection
5. Resource allocation
6. Accounting
7. protection

- Processor Management
- Device Management
- Memory Management
- File Management
- Security

TYPES OF OPERATING SYSTEM

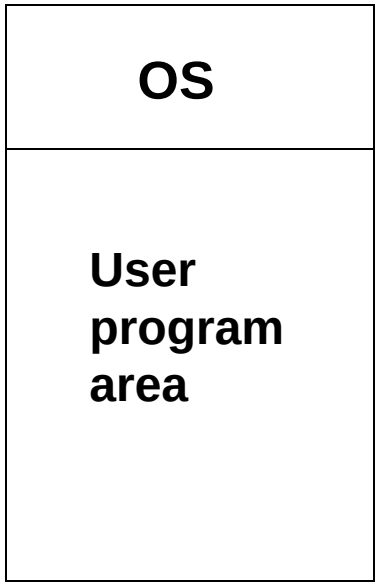
- Batch Operating System
- Time-Sharing Operating System
- Distributed Operating System
- Embedded Operating System
- Real-time Operating System

Batch Processing:

- In Batch processing same type of jobs batch *(BATCH- a set of jobs with similar needs)* together and execute at a time.
- The OS was simple, its major task was to transfer control from one job to the next.
- The job was submitted to the computer operator in form of punch cards. At some later time the output appeared.
- The OS was always resident in memory.
- Common Input devices were card readers and tape drives.

Batch Processing (Contd…):

- Common output devices were line printers, tape drives, and card punches.
- Users did not interact directly with the computer systems, but he prepared a job (comprising of the program, the data, & some control information).



Advantages:

1.The overall time taken by the system to execute all the programmes will be reduced.

2.The Batch Operating System can be shared between multiple users.

Disadvantages:

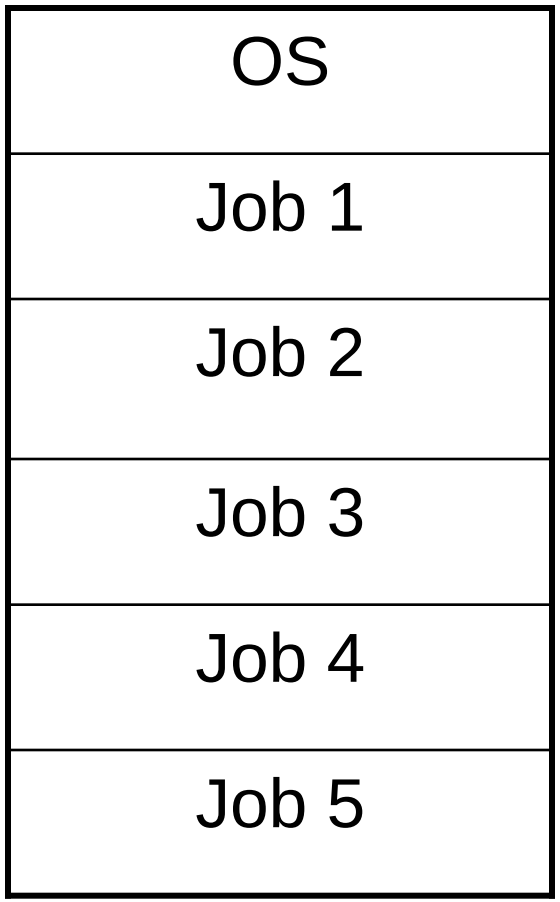
1.Manual interventions are required between two batches.

2.The CPU utilization is low because the time taken in loading and unloading of batches is very high as compared to execution time.

Multiprogramming:

- Multiprogramming is a technique to execute number of programs simultaneously by a single processor.
- In Multiprogramming, number of processes reside in main memory at a time.
- The OS picks and begins to executes one of the jobs in the main memory.
- If any I/O wait happened in a process, then CPU switches from that job to another job.
- Hence CPU in not idle at any time.

Multiprogramming (Contd…):



- Figure dipicts the layout of multiprogramming system.
- The main memory consists of 5 jobs at a time, the CPU executes one by one.

Advantages:

- Efficient memory utilization
- Throughput increases
- CPU is never idle, so performance increases.

Time Sharing Systems:

- Time sharing, or multitasking, is a logical extension of multiprogramming.
- Multiple jobs are executed by switching the CPU between them.
- In this, the CPU time is shared by different processes, so it is called as “Time sharing Systems”.
- Time slice is defined by the OS, for sharing CPU time between processes.
- Examples: Multics, Unix, etc.,

Advantages:

1.Since equal time quantum is given to each process, so each process gets equal opportunity to execute.

2.The CPU will be busy in most of the cases and this is good to have case.

Disadvantages:

1.Process having higher priority will not get the chance to be executed first because the equal opportunity is given to each process.

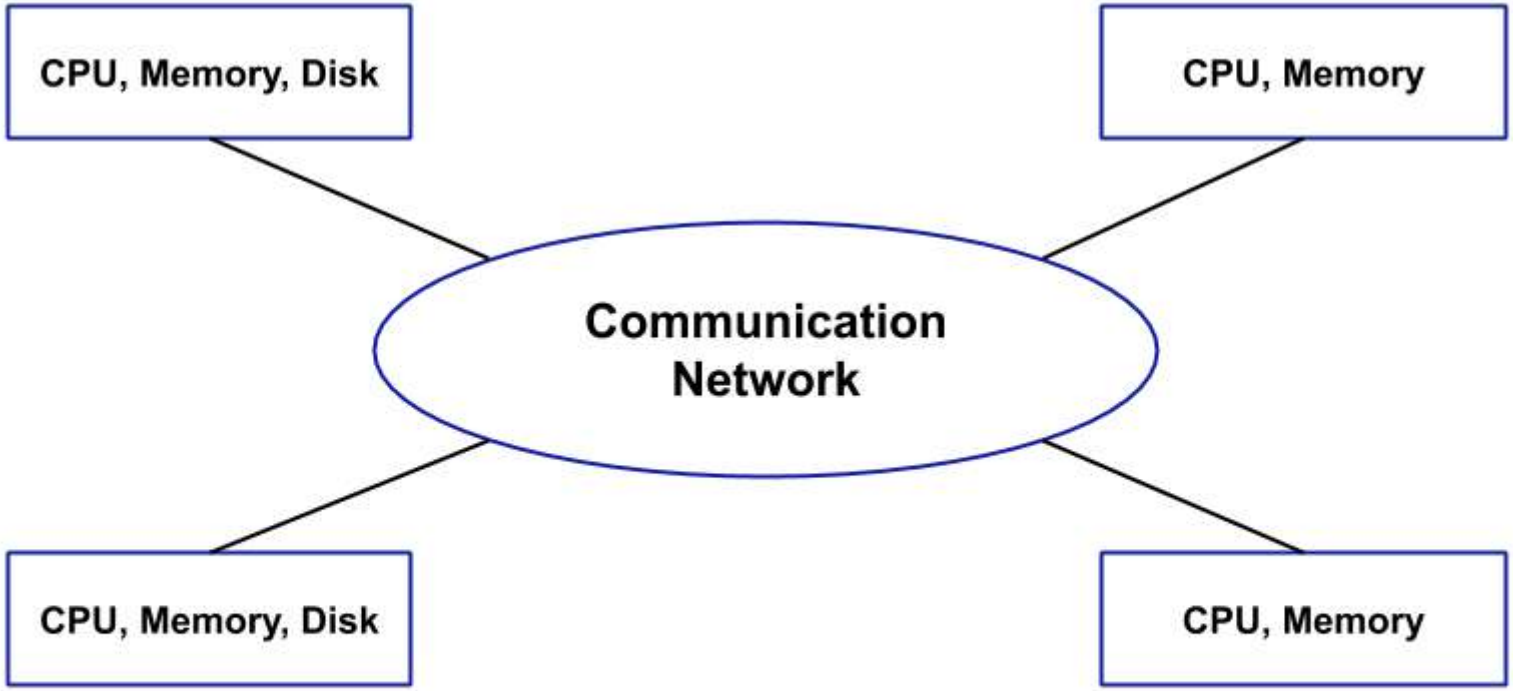
Distributed Operating System

- In a Distributed Operating System, we have various systems and all these systems have their own CPU, main memory, secondary memory, and resources.

- These systems are connected to each other using a shared communication network. Here, each system can perform its task individually.

- The best part about these Distributed Operating System is remote access i.e. one user can access the data of the other system and can work accordingly.

- So, remote access is possible in these distributed Operating Systems.



Advantages:

1.Since the systems are connected with each other so, the failure of one system can't stop the execution of processes because other systems can do the execution.

2.Resources are shared between each other.

3.The load on the host computer gets distributed and this, in turn, increases the efficiency.

Disadvantages:

1.Since the data is shared among all the computers, so to make the data secure and accessible to few computers, you need to put some extra efforts.

2.If there is a problem in the communication network then the whole communication will be broken.

Embedded Operating System

An Embedded Operating System is designed to perform a specific task for a particular device which is not a computer.

For example, the software used in elevators is dedicated to the working of elevators only and nothing else.

So, this can be an example of Embedded Operating System.

The Embedded Operating System allows the access of device hardware to the software that is running on the top of the Operating System.

Advantages:

1.Since it is dedicated to a particular job, so it is fast.

2.Low cost.

3.These consume less memory and other resources.

Disadvantages:

1.Only one job can be performed.

2.It is difficult to upgrade or is nearly scalable.

Real-Time Operating System –

These types of OSs serve real-time systems. The time interval required to process and respond to inputs is very small. This time interval is called **response time**.

Real-time systems are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Two types of Real-Time Operating System which are as follows:

- **Hard Real-Time Systems:**

These OSs are meant for applications where time constraints are very strict and even the shortest possible delay is not acceptable. These systems are built for saving life like automatic parachutes or airbags which are required to be readily available in case of any accident. Virtual memory is rarely found in these systems.

- **Soft Real-Time Systems:**

These OSs are for applications where for time-constraint is less strict.

Advantages of RTOS:

- **Maximum Consumption:** Maximum utilization of devices and system, thus more output from all the resources
- **Task Shifting:** The time assigned for shifting tasks in these systems are very less. For example, in older systems, it takes about 10 microseconds in shifting one task to another, and in the latest systems, it takes 3 microseconds.
- **Focus on Application:** Focus on running applications and less importance to applications which are in the queue.
- Real-time **operating system in** the **embedded system:** Since the size of programs are small, RTOS can also be used in embedded systems like in transport and others.
- **Error Free:** These types of systems are error-free.
- **Memory Allocation:** Memory allocation is best managed in these types of systems.

Disadvantages of RTOS:

- **Limited Tasks:** Very few tasks run at the same time and their concentration is very less on few applications to avoid errors.
- **Use heavy system resources:** Sometimes the system resources are not so good and they are expensive as well.
- **Complex Algorithms:** The algorithms are very complex and difficult for the designer to write on.
- **Device driver and interrupt signals:** It needs specific device drivers and interrupts signals to respond earliest to interrupts.
- **Thread Priority:** It is not good to set thread priority as these systems are very less prone to switching tasks.

Examples of Real-Time Operating Systems are: Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

Operating System Services

- Program execution – system capability to load a program into memory and to run it
- I/O operations – since user programs cannot execute I/O operations directly, the operating system must provide some means to perform I/O
- File-system manipulation – program capability to read, write, create, and delete files
- Communications – exchange of information between processes executing either on the same computer or on different systems tied together by a network. Implemented via *shared memory* or *message passing*
- Error detection – ensure correct computing by detecting errors in the CPU and memory hardware, in I/O devices, or in user programs

Components of Operating System

- Process Management
- Main Memory Management
- File Management
- I/O System Management
- Secondary Management
- Networking
- Protection System
- Command-Interpreter System

Process Management

- A *process* is a program in execution
  - A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task
- The operating system is responsible for the following activities in connection with process management
  - Process creation and deletion
  - Process suspension and resumption
  - Provision of mechanisms for:
    - process synchronization
    - process communication

Main-Memory Management

- *Memory* is a large array of words or bytes, each with its own address
  - It is a repository of quickly accessible data shared by the CPU and I/O devices
- *Main memory* is a volatile storage device. It loses its contents in the case of system failure
- The operating system is responsible for the following activities in connections with memory management
  - Keep track of which parts of memory are currently being used and by whom
  - Decide which processes to load when memory space becomes available
  - Allocate and deallocate memory space as needed

File Management

- A *file* is a collection of related information defined by its creator
  - Commonly, files represent programs (both source and object forms) and data
- The operating system is responsible for the following activities in connections with file management:
  - File creation and deletion
  - Directory creation and deletion
  - Support of primitives for manipulating files and directories
  - Mapping files onto *secondary storage*
  - File backup on *stable (nonvolatile) storage* media

I/O System Management

- The I/O system consists of:
  - A buffer-caching system
  - A general device-driver interface
  - Drivers for specific hardware devices

Secondary-Storage Management

- Since main memory (*primary storage*) is volatile and too small to accommodate all data and programs permanently, the computer system must provide *secondary storage* to back up main memory
- Most modern computer systems use disks as the principle on-line storage medium, for both programs and data
- The operating system is responsible for the following activities in connection with disk management:
  - Free space management
  - Storage allocation
  - Disk scheduling

Networking (Distributed Systems)

- A *distributed* system is a collection processors that do not share memory or a clock
  - Each processor has its own local memory
- The processors in the system are connected through a communication network
- Communication takes place using a *protocol*
- A distributed system provides user access to various system resources
- Access to a shared resource allows:
  - Computation speed-up
  - Increased data availability
  - Enhanced reliability

Protection System

- *Protection* refers to a mechanism for controlling access by programs, processes, or users to both system and user resources
- The protection mechanism must:
  - distinguish between authorized and unauthorized usage
  - specify the controls to be imposed
  - provide a means of enforcement

Command-Interpreter System

- Many commands are given to the operating system by control statements which deal with:
  - Process creation and management
  - I/O handling
  - Secondary-storage management
  - Main-memory management
  - File-system access
  - Protection
  - Networking

Command-Interpreter System (Cont.)

- The program that reads and interprets control statements is called variously:
  - *command-line interpreter*
  - *shell (in UNIX)*

  Its function is to get and execute the next command statement

Additional Operating System Functions

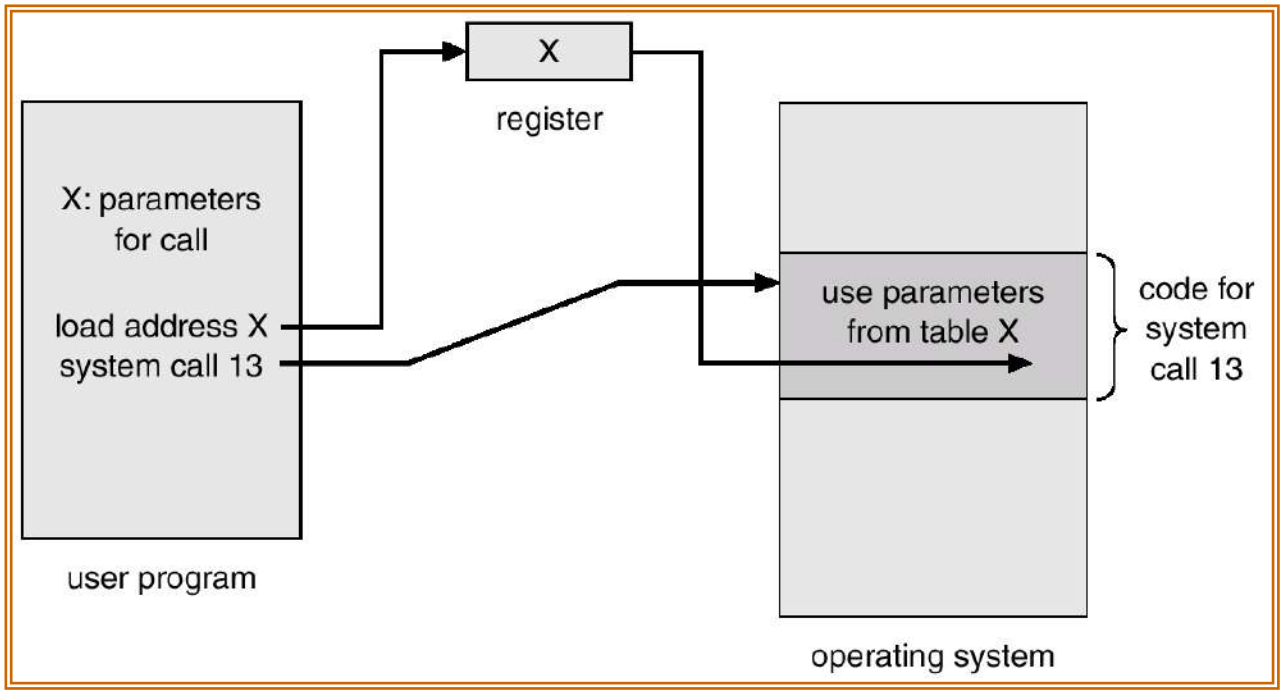
Additional functions exist not for helping the user, but rather for ensuring efficient system operations

- Resource allocation – allocating resources to multiple users or multiple jobs running at the same time
- Accounting – keep track of and record which users use how much and what kinds of computer resources for account billing or for accumulating usage statistics
- Protection – ensuring that all access to system resources is controlled

System Calls

- *System calls* provide the interface between a running program and the operating system
  - Generally available as assembly-language instructions
  - Languages defined to replace assembly language for systems programming allow system calls to be made directly (e.g., C, C++)
- Three general methods are used to *pass parameters* between a running program and the operating system
  - Pass parameters in *registers*
  - Store the parameters in a table in memory, and the table address is passed as a parameter in a register
  - *Push* (store) the parameters onto the *stack* by the program, and *pop* off the stack by operating system

Passing of Parameters As A Table



Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications

- Process control
  - Create,process,terminate process
  - Load,execute a process
  - End, abort a process
  - Get procee attributes, set process attributes
  - Wait for time, wait event, signal event
  - Allocate, free memory
- File Management
  - Create file, delete file
  - Open,close a file
  - Read, write a file
  - Get file attributes, set file attributes

- Device Management
  - Request device, release device
  - Read, write, reposition
  - Get device attribute, set device attributes
- Information Maintenance
  - Get time or data, set time or date
  - Get system data, Set system data
  - Get process, file or device attributes
  - Set process, file or device attributes
- Communication Maintenance
  - Create, delete communication connection
  - Send, receive messages
  - transfer status information
  - attach or detach remote devices

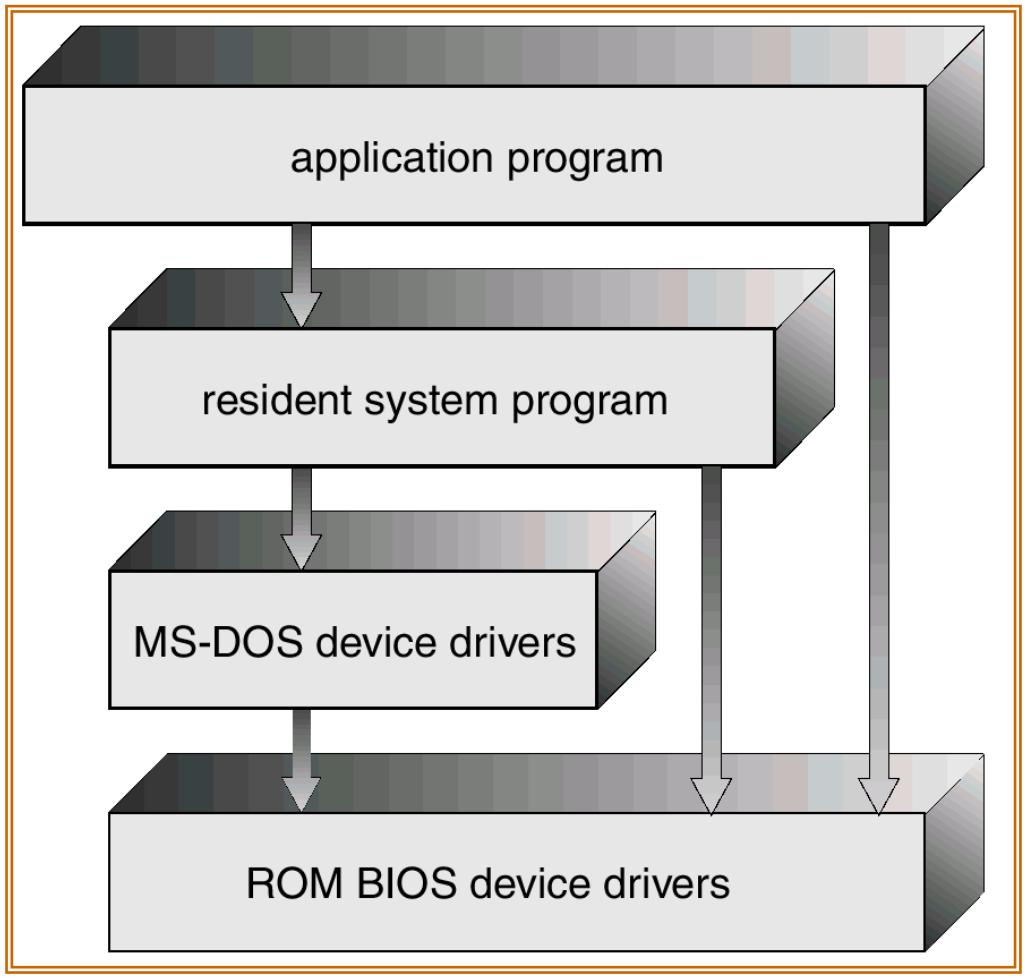
System Programs

- System programs provide a convenient environment for program development and execution. The can be divided into:
  - File manipulation
  - Status information
  - File modification
  - Programming language support
  - Program loading and execution
  - Communications
  - Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls

MS-DOS System Structure

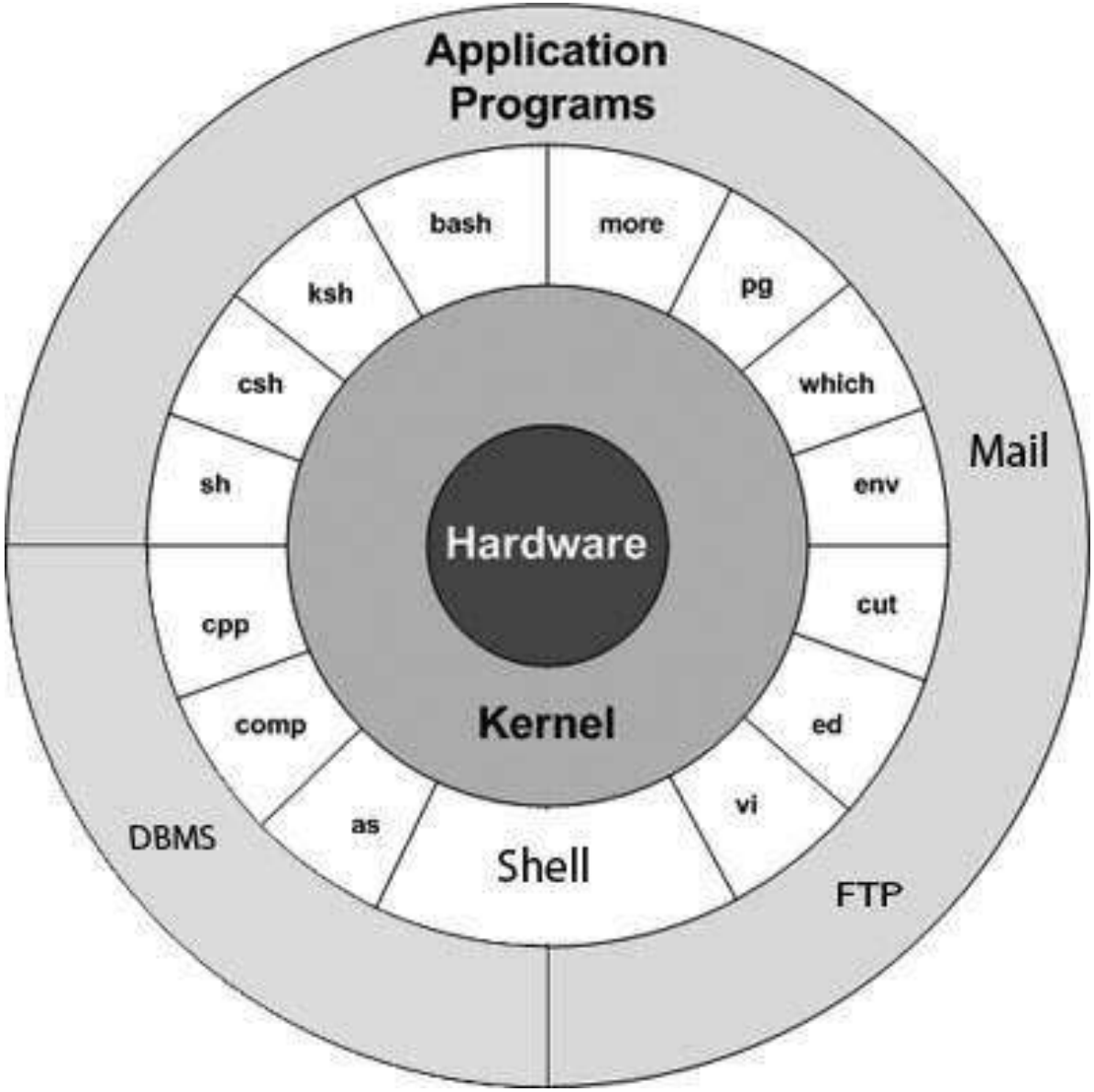
- MS-DOS – written to provide the most functionality in the least space
  - Not divided into modules
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated

MS-DOS Layer Structure



UNIX System Structure

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts
  - Systems programs
  - The kernel
    - Consists of everything below the system-call interface and above the physical hardware
    - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level


Application Programs
bash
more
pg
which
env
cut
ed
vi
as
comp
cpp
sh
csh
ksh
Hardware
Kernel
Shell
Mail
FTP
DBMS

•**Shell** − The shell is the utility that processes our requests. When we type in a command at our terminal, the shell interprets the command and calls the program that we want. The shell uses standard syntax for all commands. C Shell, Bourne Shell and Korn Shell are the most famous shells which are available with most of the Unix variants.

•**Commands and Utilities** − There are various commands and utilities which we can make use of in our day to day activities. **cp**, **mv**, **cat** and **grep**, etc. are few examples of commands and utilities. There are over 250 standard commands plus numerous others provided through 3rd party software. All the commands come along with various options.

•**Files and Directories** − All the data of Unix is organized into files. All files are then organized into directories. These directories are further organized into a tree-like structure called the **filesystem**.

Operating System and Unix

Dr Hamela K

Assistant Professor and Head

Department of Computer Science

GFGC, Malur

Chapter 1: Cont..

- **Unit-I**
- Introduction: Definition, Types of Operating Systems, Functions of Operating System, services, system components System call. Process Management: Process Concept, Process Scheduling, Inter process communication, CPU Scheduling Criteria, Scheduling algorithm, Multiple Processor Scheduling, Real time Scheduling, Algorithm evolution

Process Concept

- An operating system executes a variety of programs:
  - Batch system – **jobs**
  - Time-shared systems – **user programs** or **tasks**
- Textbook uses the terms ***job*** and ***process*** almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion
- Multiple parts
  - The program code, also called **text section**
  - Current activity including **program counter**, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time

Process Concept (Cont.)

- Program is ***passive*** entity stored on disk (**executable file**),

  process is ***active***
  - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
  - Consider multiple users executing the same program

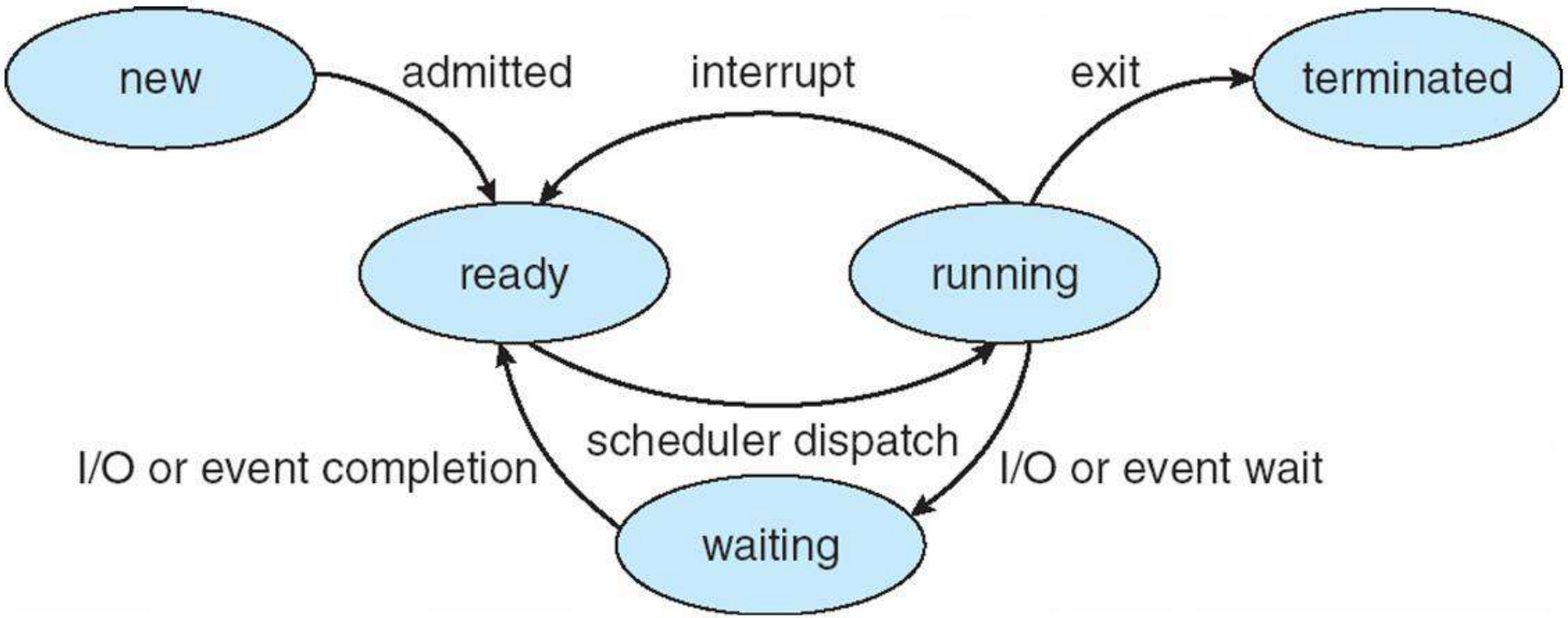
S.No.	Program	Process
1	It is a set of instruction written to solve a problem.	It is a program in execution.
2	It is a passive entity such as a file stored on a disk.	It is an active entity as a number of resources are associated with it.
3	A program is a simple entity in itself.	Two or more process may be associated with the same program.
4	Programs are present usually in the secondary memory.	Processes are found in the main memory.
5	Program remains in same state throughout its life time.	A process continues to change its state among five possible states new, running, ready, waiting and terminated. Each state indicates current activity of process.
6	A program is represented on the disk by its name.	A process is represented in main memory by PCB or process control block.
7	Numbers of programs on disk are limited by size of disk.	Number of processes in main memory is limited by size of main memory.

@https://computerganga.com/program-and-process/

Process State

- As a process executes, it changes **state**
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution

Diagram of Process State



Process Control Block (PCB)

Information associated with each process

(also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

Operating System and Unix

Dr Hamela K

Assistant Professor and Head

Department of Computer Science

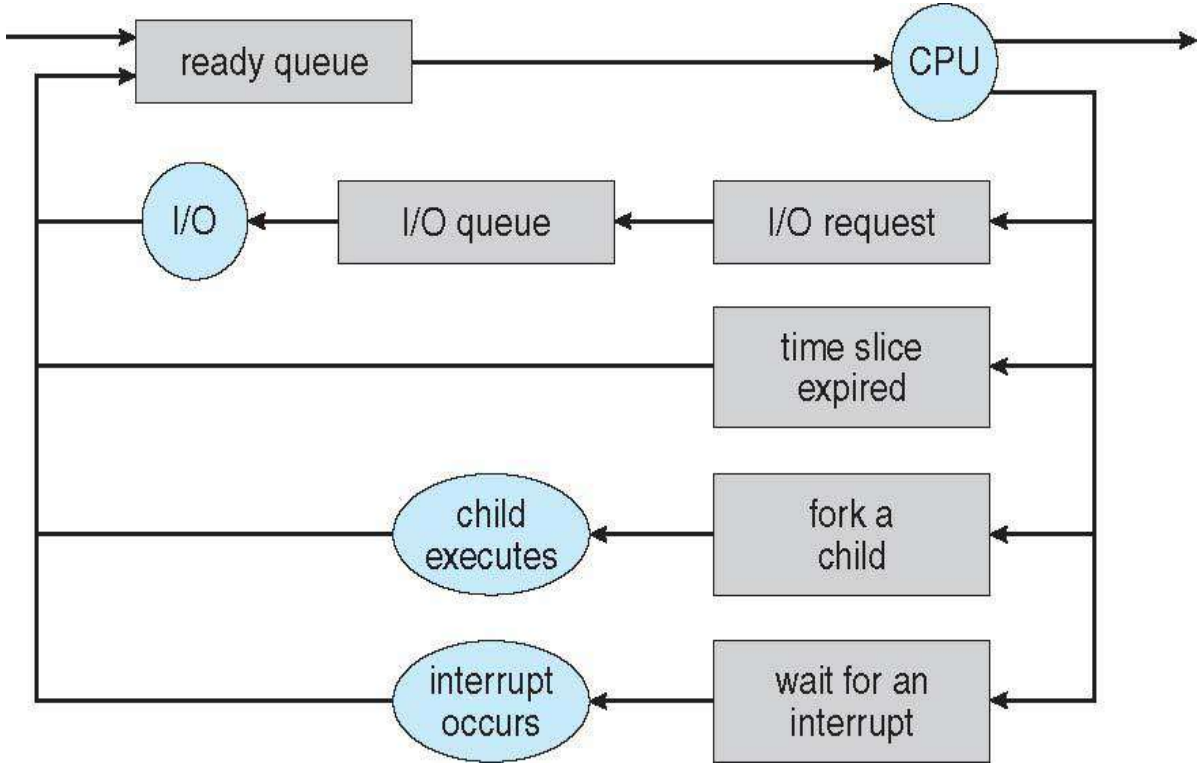
GFGC, Malur

Process Scheduling

- The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.
- Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Process Scheduling

- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues



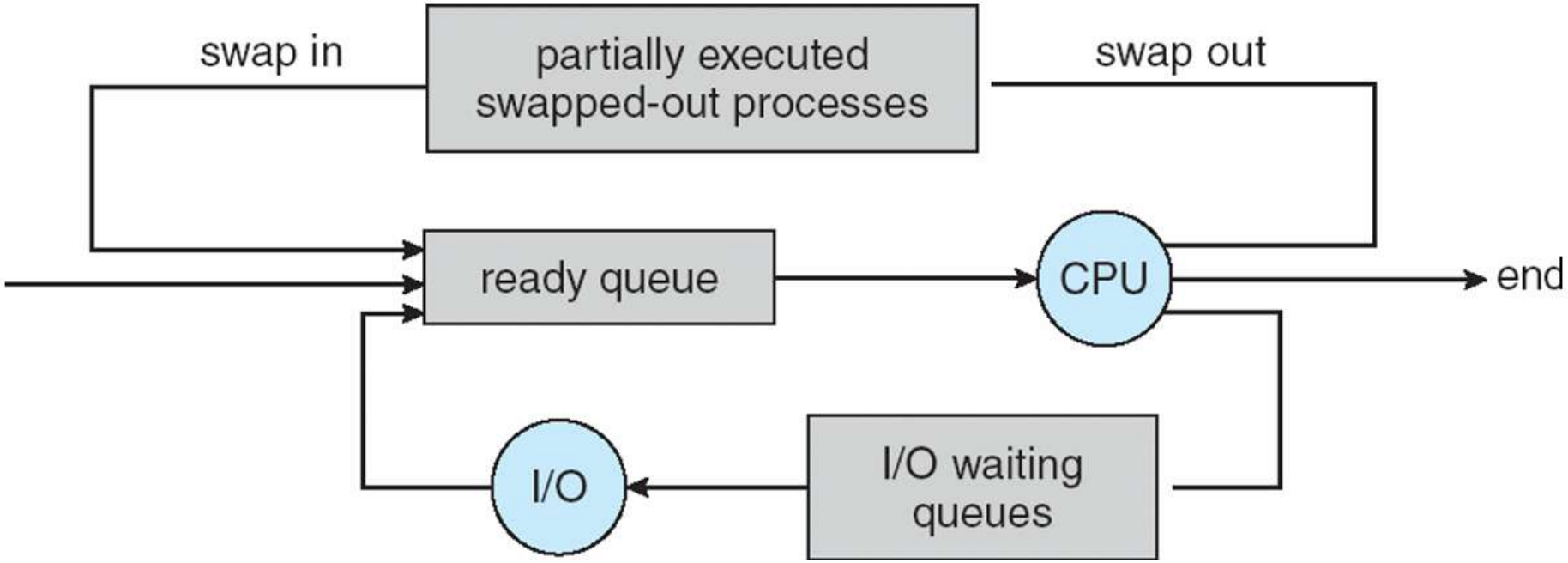
Schedulers

The selection of process and its corresponding queue is done by the part of the operating system called Scheduler

- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - Sometimes the only scheduler in a system
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***

Medium Term Scheduling

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
  - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**



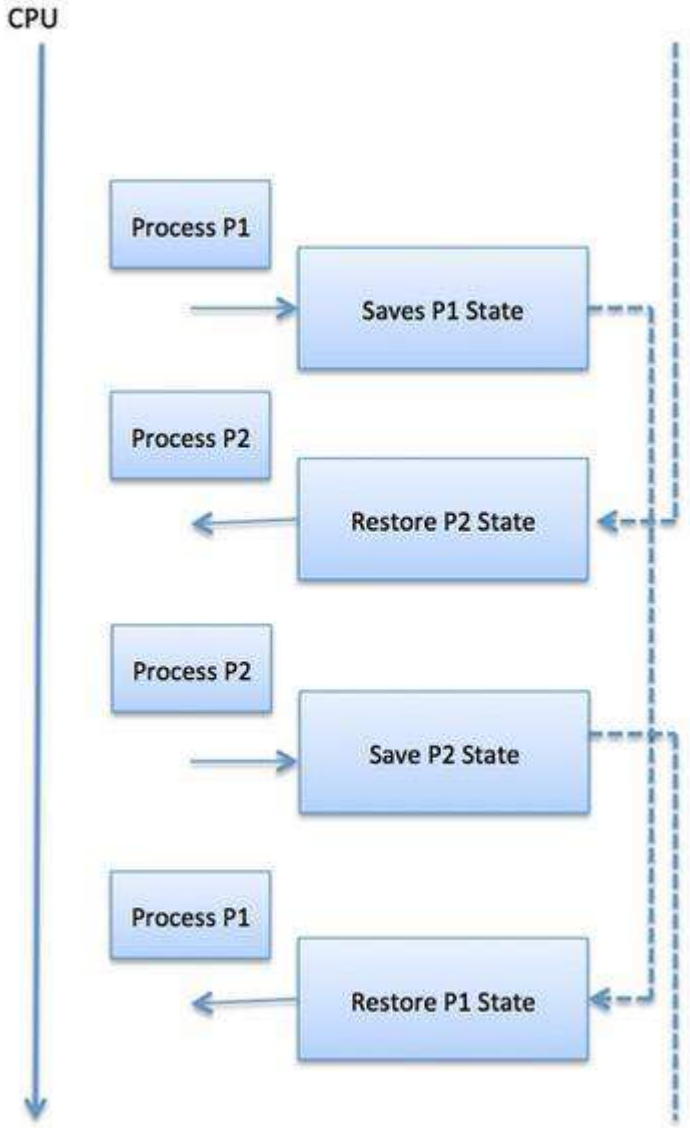
Comparison among Scheduler

S.N.	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming.
4	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.
5	It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

Context Switch

A context switch is the mechanism to store and restore the state or context of a CPU in Process Control block so that a process execution can be resumed from the same point at a later time. Using this technique, a context switcher enables multiple processes to share a single CPU. Context switching is an essential part of a multitasking operating system features.

When the scheduler switches the CPU from executing one process to execute another, the state from the current running process is stored into the process control block. After this, the state for the process to run next is loaded from its own PCB and used to set the PC, registers, etc. At that point, the second process can start executing.



Interprocess communication

- Interprocess communication is the mechanism provided by the operating system that allows processes to communicate with each other.
- This communication could involve a process letting another process know that some event has occurred or the transferring of data from one process to another.

BASICS OF INTERPROCESSES COMMUNICATION

There are numerous reasons for providing an environment or situation which allows process co-operation:

- **Information sharing:** Since some users may be interested in the same piece of information (for example, a shared file), you must provide a situation for allowing concurrent access to that information.
- **Computation speedup:** If you want a particular work to run fast, you must break it into sub-tasks where each of them will get executed in parallel with the other tasks.
- **Modularity:** You may want to build the system in a modular way by dividing the system functions into split processes or threads.
- **Convenience:** Even a single user may work on many tasks at a time. For example, a user may be editing, formatting, printing, and compiling in parallel

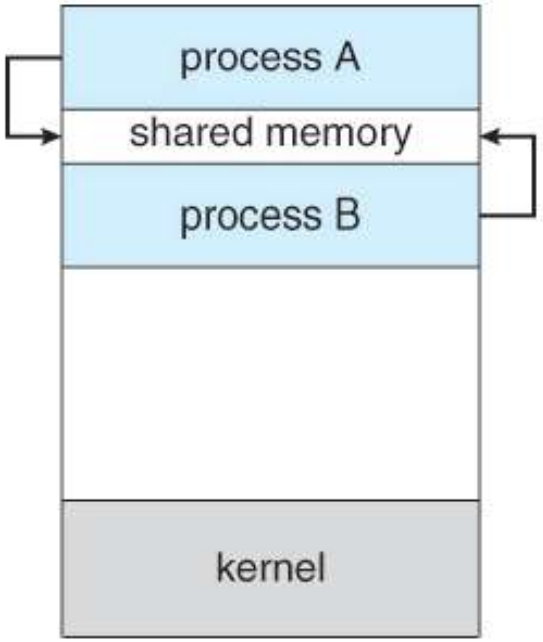
Working together with multiple processes, require an interprocess communication (IPC) method which will allow them to exchange data along with various information. There are two primary models of interprocess communication:

1.shared memory and

2.message passing.

SHARED MEMORY

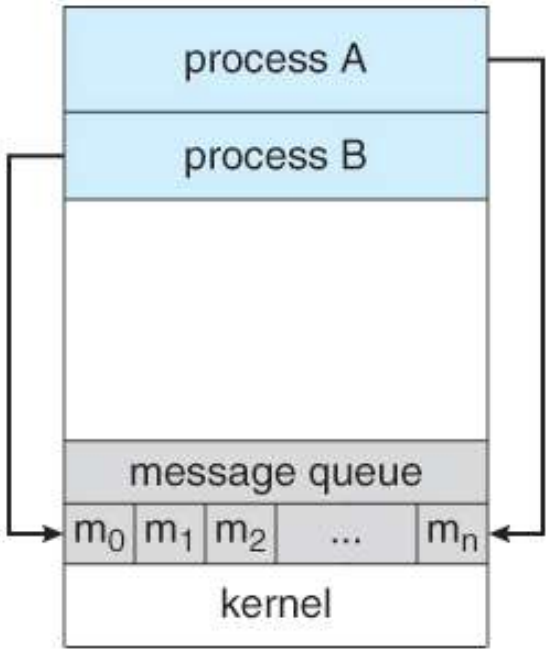
- In the shared-memory model, a region of memory which is shared by cooperating processes gets established. Processes can be then able to exchange information by reading and writing all the data to the shared region.
- Interprocess communication (IPC) usually utilizes shared memory that requires communicating processes for establishing a region of shared memory. Typically, a shared-memory region resides within the address space of any process creating the shared memory segment. Other processes that wish for communicating using this shared-memory segment must connect it to their address space.



(b) Shared Memory

- Shared Memory
- A particular region of memory is shared between cooperating process.
- Cooperating process can exchange information by reading and writing data to this shared region.
- It’s faster than Memory Parsing, as Kernel is required only once, that is, setting up a shared memory . After That, kernel assistance is not required.

MESSAGE-PASSING



(a) Message Passing

In the message-passing form, communication takes place by way of messages exchanged among the cooperating processes.

In this method, processes communicate with each other without using any kind of shared memory. If two processes p1 and p2 want to communicate with each other, they proceed as follows:

Usually, the inter-process communication mechanism provides two operations that are as follows:

- send (message)
- received (message)

Direct Communication:-

In this type of communication process, usually, a link is created or established between two communicating processes. However, in every pair of communicating processes, only one link can exist.

Indirect Communication

Indirect communication can only exist or be established when processes share a common mailbox, and each pair of these processes shares multiple communication links. These shared links can be unidirectional or bi-directional.

Message Parsing

1.Communication takes place by exchanging messages directly between cooperating process.
2.Easy to implement
3.Useful for small amount of data.
4.Implemented using System Calls, so takes more time than Shared Memory.

Cooperating Processes

- ***Independent*** process cannot affect or be affected by the execution of another process
- ***Cooperating*** process can affect or be affected by the execution of another process
- Advantages of process cooperation
  - Information sharing
  - Computation speed-up
  - Modularity
  - Convenience

CPU Scheduling in Operating System

- CPU scheduling is a process that allows one process to use the CPU while the execution of another process is on hold(in waiting state) due to unavailability of any resource like I/O etc, thereby making full use of CPU.
- The aim of CPU scheduling is to make the system efficient, fast, and fair.

CPU Scheduling in Operating System

CPU Scheduling: Dispatcher

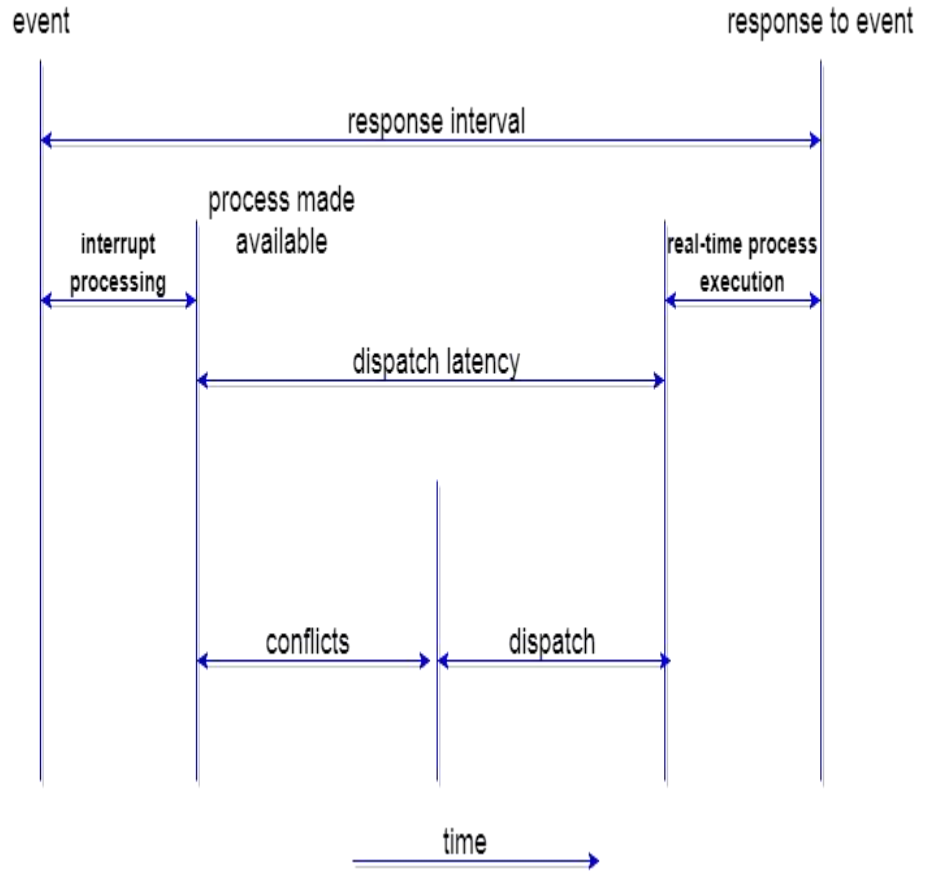
Another component involved in the CPU scheduling function is the **Dispatcher**. The dispatcher is the module that gives control of the CPU to the process selected by the **short-term scheduler**. This function involves:

Switching context

Switching to user mode

Jumping to the proper location in the user program to restart that program from where it left last time.

The dispatcher should be as fast as possible, given that it is invoked during every process switch. The time taken by the dispatcher to stop one process and start another process is known as the **Dispatch Latency**. Dispatch Latency can be explained using the below figure:



Types of CPU Scheduling

CPU scheduling decisions may take place under the following four circumstances:

1.When a process switches from the **running** state to the **waiting** state(for I/O request or invocation of wait for the termination of one of the child processes).

2.When a process switches from the **running** state to the **ready** state (for example, when an interrupt occurs).

3.When a process switches from the **waiting** state to the **ready** state(for example, completion of I/O).

4.When a process **terminates**.

In circumstances 1 and 4, there is no choice in terms of scheduling. A new process(if one exists in the ready queue) must be selected for execution. There is a choice, however in circumstances 2 and 3.

When Scheduling takes place only under circumstances 1 and 4, we say the scheduling scheme is **non-preemptive**; otherwise, the scheduling scheme is **preemptive**.

Non-Preemptive Scheduling

Under non-preemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.

Preemptive Scheduling is a scheduling method where the tasks are mostly assigned with their priorities. Sometimes it is important to run a task with a higher priority before another lower priority task, even if the lower priority task is still running.

CPU Scheduling: Scheduling Criteria

CPU Utilization

To make out the best use of the CPU and not to waste any CPU cycle, the CPU would be working most of the time(Ideally 100% of the time). Considering a real system, CPU usage should range from 40% (lightly loaded) to 90% (heavily loaded.)

Throughput

It is the total number of processes completed per unit of time or rather says the total amount of work done in a unit of time. This may range from 10/second to 1/hour depending on the specific processes.

Turnaround Time

It is the amount of time taken to execute a particular process, i.e. The interval from the time of submission of the process to the time of completion of the process(Wall clock time).

Waiting Time

The sum of the periods spent waiting in the ready queue amount of time a process has been waiting in the ready queue to acquire get control on the CPU.

Load Average

It is the average number of processes residing in the ready queue waiting for their turn to get into the CPU.

Response Time

Amount of time it takes from when a request was submitted until the first response is produced. Remember, it is the time till the first response and not the completion of process execution(final response).

In general CPU utilization and Throughput are maximized and other factors are reduced for proper optimization.

SESSION OVERVIEW

- PROCESS CONCEPT
- PROCESS SCHEDULING
- INTERPROCESS COMMUNICATION
- CPU SCHEDULING CRITERIA

CPU SCHEDULING ALGORITHM

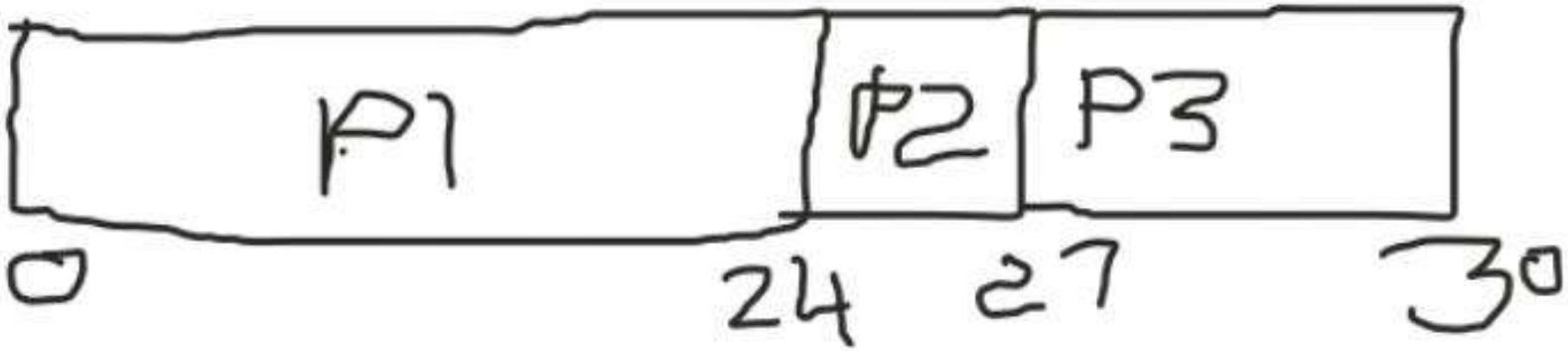
There are popular process scheduling algorithms −

- First-Come, First-Served (FCFS) Scheduling
- Shortest-Job-Next (SJN) Scheduling
- Priority Scheduling
- Round Robin(RR) Scheduling
- Multiple-Level Queues Scheduling
- Multilevel Feedback Queue Scheduling

First Come First Serve (FCFS)

- Jobs are executed on first come, first serve basis.
- It is a non-preemptive, pre-emptive scheduling algorithm.
- Easy to understand and implement.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

GANTT CHART



PROCESS	CPU TIME
P1	24
P2	3
P3	3

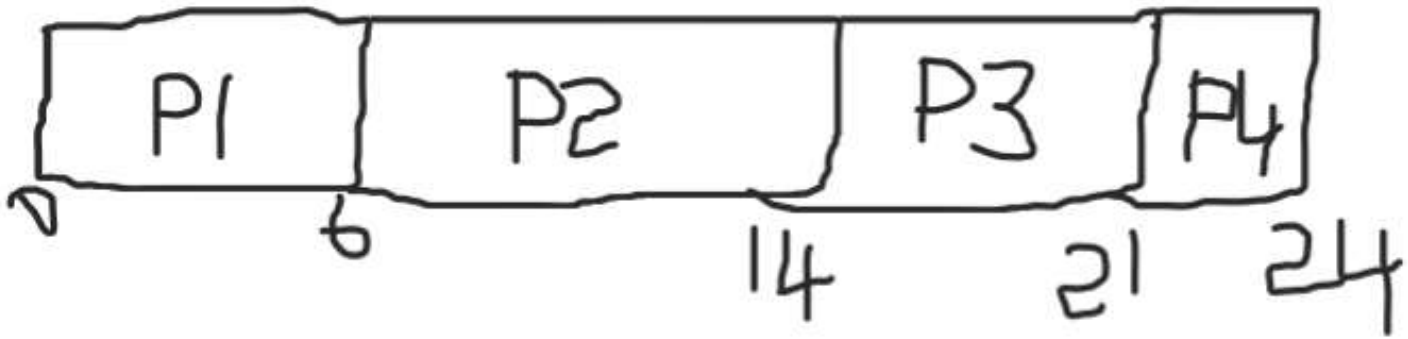
Waiting Time
Waiting time of p1=0-0=0
Waiting time of p2=24-0=24
Waiting time of p3=27-0=27
Avg Waiting Time=17ms

Response Time
Response Time of p1=0-0=0
Response Time of p2=24-0=24
Response Time of p3=27-0=27
Avg Response Time=17ms

Turnaround Time
Turnaround Time of p1=24-0=24
Turnaround Time of p2=27-0=27
Turnaround Time of p3=30-0=30
Avg Throughput Time =27ms

Throughput=3/30=0.1ms

GANTT CHART



PROCESS	CPU TIME
P1	6
P2	8
P3	7
P4	3

Waiting Time
Waiting time of p1=0-0=0
Waiting time of p2=6-0=6
Waiting time of p3=14-0=14
Waiting time of p4=21-0=21
Avg Waiting Time=10.25ms

Response Time
Response Time of p1=0-0=0
Response Time of p2=6-0=6
Response Time of p3=14-0=14
Response Time of p4=21-0=21
Avg Response Time=10.25ms

Turnaround Time
Turnaround Time of p1=6-0=6
Turnaround Time of p2=14-0=14
Turnaround Time of p3=21-0=21
Turnaround Time of p4=24-0=24
Avg Throughput Time =16.25ms

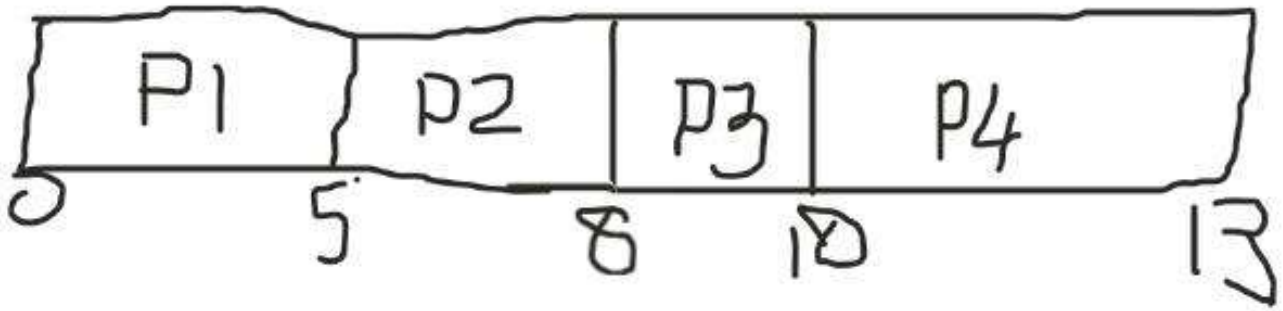
Throughput=4/24=0.166ms

FCFS – NON PREEMPTIVE SCHEDULING: No process is interrupted until it is completed, and after that processor switches to another process.

FCFS – PREEMPTIVE SCHEDULING: This scheduling is used when the process switch to ready state.

FCFS SCHEDULING USING PREEMPTIVE SCHEDULING

GANTT CHART



PROCESS	CPU TIME	ARRIVAL TIME
P1	5	0
P2	3	2
P3	2	6
P4	3	7

Waiting Time
Waiting time of p1=0-0=0
Waiting time of p2=5-2=3
Waiting time of p3=8-6=2
Waiting time of p4=10-7=3
Avg Waiting Time=2ms

Response Time
Response Time of p1=0-0=0
Response Time of p2=5-2=3
Response Time of p3=8-6=2
Response Time of p4=10-7=3
Avg Response Time=2ms

Turnaround Time
Turnaround Time of p1=5-0=5
Turnaround Time of p2=8-2=6
Turnaround Time of p3=10-6=4
Turnaround Time of p4=13-7=6
Avg Throughput Time =5.25ms

Throughput=4/13=0.30ms

HOMEWORK: FCFS SCHEDULING USING PREEMPTIVE SCHEDULING

1.

Process	Arrival time	Burst time
P1	0 ms	18 ms
P2	2 ms	7 ms
P3	2 ms	10 ms

2.

Process	Arrival Time	Execute Time	Service Time
P0	0	5	0
P1	1	3	5
P2	2	8	8
P3	3	6	16

Shortest job first (SJF) or shortest job next, is a scheduling policy that selects the waiting process with the smallest execution time to execute next. SJN is a non-preemptive algorithm.

- Shortest Job first has the advantage of having a minimum average waiting time among all scheduling algorithms.
- It is a Greedy Algorithm.
- It may cause starvation if shorter processes keep coming. This problem can be solved using the concept of ageing.
- It is practically infeasible as Operating System may not know burst time and therefore may not sort them.

Algorithm:

1. Sort all the process according to the arrival time.

2. Then select that process which has minimum arrival time and minimum Burst time.

3. After completion of process make a pool of process which after till the completion of previous process and select that process among the pool which is having minimum Burst time.

Advantages of SJF

1.Maximum throughput

2.Minimum average waiting and turnaround time

Disadvantages of SJF

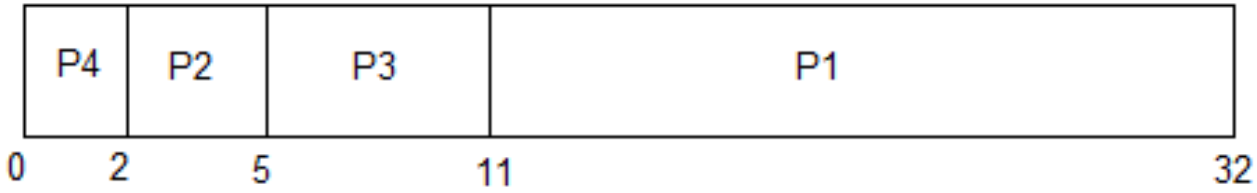
1.May suffer with the problem of starvation

2.It is not implementable because the exact Burst time for a process can't be known in advance

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2

In Shortest Job First Scheduling, the shortest Process is executed first.

Hence the GANTT chart will be following :



Now, the average waiting time will be = (0 + 2 + 5 + 11)/4 = 4.5 ms

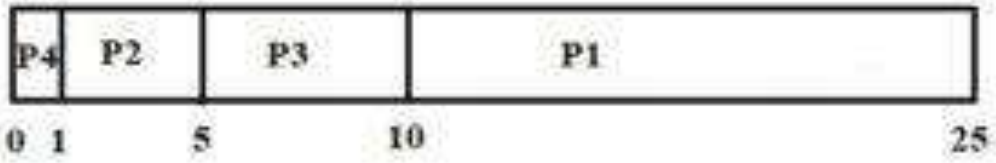
Average Turn around time:
For p1:32-0=32
For p2:5-0=5
For p3:11-0=11
For p4:2-0=2
Avg turn around time=12.5ms

Average Response time = 4.5ms

Throughput=4/32 =0.125ms

Process	Burst Time
P1	15
P2	4
P3	5
P4	1

In Shortest Job First Scheduling. the shortest process is executed first.



GANTT Chart for above Process

$$\text{Average Waiting Time} : \frac{0 + 1 + 5 + 10}{4} = 4 \text{ ms}$$

Average Turn around time:
For p1:25-0=25
For p2:5-0=5
For p3:10-0=10
For p4:1-0=1
Avg turn around time=10.5ms

Average Response time = 4ms

Throughput=4/25 =0.16ms

HOME WORK USING SJF ALGORITHM

Using SJF algorithm calculate waiting time, response time, turnaround time and througput

Process	Burst Time (mills.)
P1	9
P2	4
P3	5
P4	7
P5	3

Process	Burst Time
P1	10
P2	5
P3	8

Priority Scheduling

- **Priority Scheduling** is a method of scheduling processes that is based on priority. In this algorithm, the scheduler selects the tasks to work as per the priority.
- The processes with higher priority should be carried out first, whereas jobs with equal priorities are carried out on a round-robin or FCFS basis. Priority depends upon memory requirements, time requirements, etc.

Types of Priority Scheduling Algorithm

Priority scheduling can be of two types:

1. **Preemptive Priority Scheduling**: If the new process arrived at the ready queue has a higher priority than the currently running process, the CPU is preempted, which means the processing of the current process is stoped and the incoming new process with higher priority gets the CPU for its execution.

2. **Non-Preemptive Priority Scheduling**: In case of non-preemptive priority scheduling algorithm if a new process arrives with a higher priority than the current running process, the incoming process is put at the head of the ready queue, which means after the execution of the current process it will be processed.

- **Characteristics of Priority Scheduling**
- A CPU algorithm that schedules processes based on priority.
- It used in Operating systems for performing batch processes.
- If two jobs having the same priority are READY, it works on a FIRST COME, FIRST SERVED basis.
- In priority scheduling, a number is assigned to each process that indicates its priority level.
- Lower the number, higher is the priority.
- In this type of scheduling algorithm, if a newer process arrives, that is having a higher priority than the currently running process, then the currently running process is preempted.

- Problem with Priority Scheduling Algorithm
- In priority scheduling algorithm, the chances of **indefinite blocking** or **starvation**.
- A process is considered blocked when it is ready to run but has to wait for the CPU as some other process is running currently.

Priority Scheduling – Eg1

process	CPU Burst time	Priority
P1	8	3
P2	4	1
P3	6	4
P4	2	2

P2	P4	P1	P3

Waiting Time: Time of Submission – Arrival Time

Waiting time of P1 = 6-0 =6
Waiting time of P2 = 0-0 =0
Waiting time of P3 = 14-0 =14
Waiting time of P4 = 4-0 =4
Average waiting time= (6+0+14+4)/4=6ms

Response Time: Time of first response/first submission-Arrival Time

Response time of P1 = 6-0 =6
Response time of P2 = 0-0 =0
Response time of P3 = 14-0 =14
Response time of P4 = 4-0 =4
Average Response time= (6+0+14+4)/4=6ms

Turn around time = Time of completion – Arrival time

Turn around time of P1 = 14-0 =14
Turn around time of P2 = 4-0 =4
Turn around time of P3 = 20-0 =20
Turn around time of P4 = 6-0 =6
Average Turn around time= (14+4+20+6)/4=44ms

Throughput= Number of processes/completion time
= 4/20=0.2ms

Priority Scheduling – Home work

PROCESS	BURST TIME	PRIORITY
P1	21	2
P2	3	1
P3	6	4
P4	2	3

The GANTT chart for following processes based on Priority scheduling will be,

Round Robin Scheduling

Round Robin is a CPU scheduling algorithm where each process is assigned a fixed time slot in a cyclic way.

- Round robin is a pre-emptive algorithm
- The CPU is shifted to the next process after fixed interval time, which is called time quantum/time slice.
- The process that is pre-empted is added to the end of the queue.
- Round robin is a hybrid model which is clock-driven
- Time slice should be minimum, which is assigned for a specific task that needs to be processed. However, it may differ OS to OS.
- It is a real time algorithm which responds to the event within a specific time limit.
- Round robin is one of the oldest, fairest, and easiest algorithm.
- Widely used scheduling method in traditional OS.

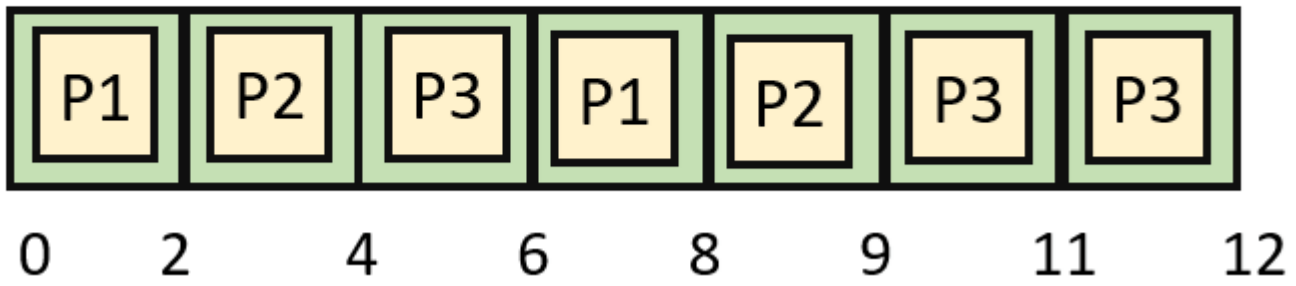
S.NO	ADVANTAGES	DISADVANTAGES
1.	There is fairness since every process gets equal share of CPU.	There is Larger waiting time and Response time.
2.	The newly created process is added to end of ready queue.	There is Low throughput.
3.	A round-robin scheduler generally employs time-sharing, giving each job a time slot or quantum.	There is Context Switches.
4.	While performing a round-robin scheduling, a particular time quantum is allotted to different jobs.	Gantt chart seems to come too big (if quantum time is less for scheduling. For Example:1 ms for big scheduling.)
5.	Each process get a chance to reschedule after a particular quantum time in this scheduling.	Time consuming scheduling for small quantums .

Example of Round-robin Scheduling

Consider this following three processes

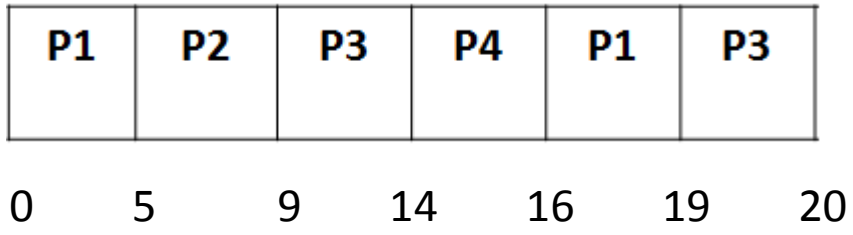
Process Queue	Burst time
P1	4
P2	3
P3	5

Time Slice = 2



RR Scheduling – Eg1

process	CPU Burst time
P1	8
P2	4
P3	6
P4	2



TIMESLICE =5 MS

Waiting Time: Time of Submission – Arrival Time

Waiting time of P1 = (0-0)+(16-5) =11
Waiting time of P2 = 5-0 =5
Waiting time of P3 = (9-0)+(19-14) =14
Waiting time of P4 = 14-0 =4
Average waiting time= (11+5+14+14)/4=44ms

Response Time: Time of first response/first submission-Arrival Time

Response time of P1 = 0-0 =0
Response time of P2 = 5-0 =5
Response time of P3 = 9-0 =9
Response time of P4 = 14-0 =14
Average Response time= (0+5+9+14)/4=7ms

Turn around time = Time of completion – Arrival time

Turn around time of P1 = 19-0 =19
Turn around time of P2 = 9-0 =9
Turn around time of P3 = 20-0 =20
Turn around time of P4 = 16-0 =16
Average Turn around time= (19+9+20+16)/4=64ms

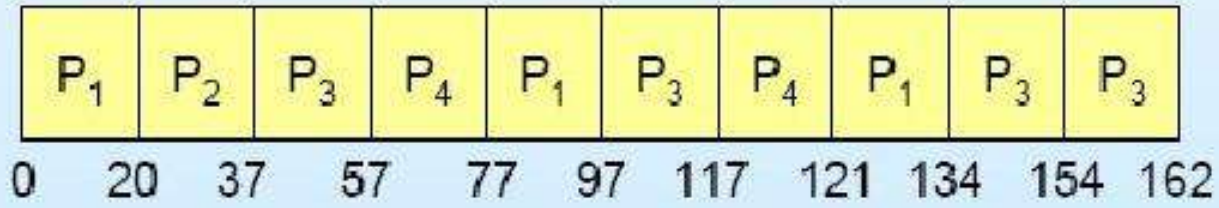
Throughput= Number of processes/completion time
= 4/20=0.2ms

HOME WORK

Round-Robin Scheduling

Process	Burst Time
P_1	53
P_2	17
P_3	68
P_4	24

The Gantt chart is:



QUATNUM TIME – 20MS

@Os.. (slideshare.net)

Home Work

Round Robin

Process	Burst Time
P1	24
P2	3
P3	3

- Quantum time = 4 milliseconds

- **A multi-level queue scheduling algorithm** partitions the ready queue into several separate queues.
- The processes are permanently assigned to one queue, generally based on some property of the process, such as memory size, process priority, or process type.
- Each queue has its own scheduling algorithm.

For example, separate queues might be used for foreground and background processes. The foreground queue might be scheduled by the Round Robin algorithm, while the background queue is scheduled by an FCFS algorithm. In addition, there must be scheduling among the queues, which is commonly implemented as fixed-priority preemptive scheduling.

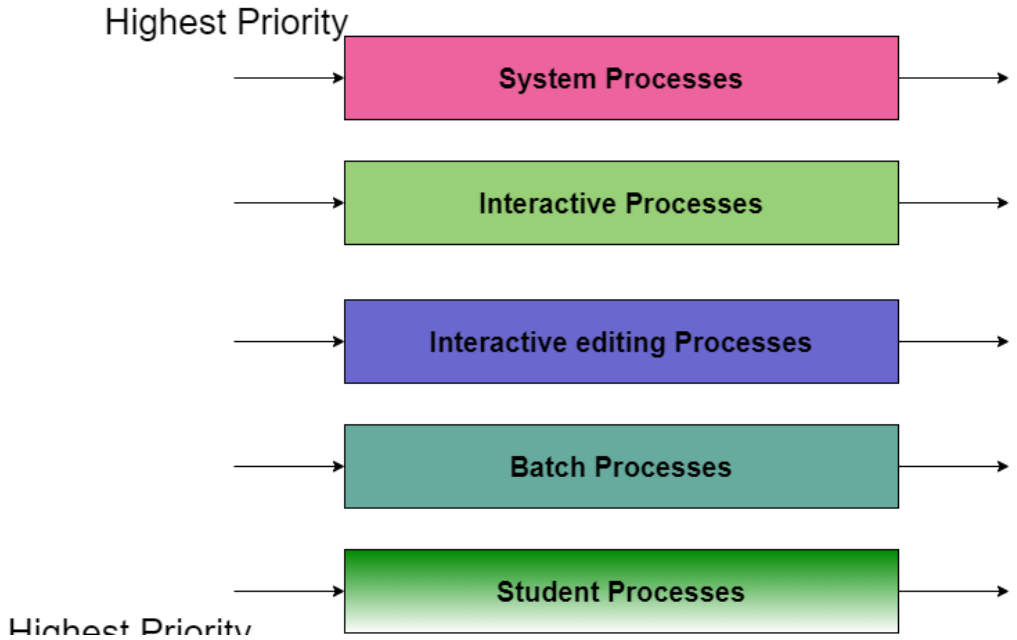
For example, The foreground queue may have absolute priority over the background queue.

Let us consider an example of a multilevel queue-scheduling algorithm with five queues:

1.System Processes
2.Interactive Processes
3.Interactive Editing Processes
4.Batch Processes
5.Student Processes

Each queue has absolute priority over lower-priority queues. No process in the batch queue, for example, could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty. If an interactive editing process entered the ready queue while a batch process was running, the batch process will be preempted.

In this case, if there are no processes on the higher priority queue only then the processes on the low priority queues will run. For **Example:** Once processes on the system queue, the Interactive queue, and Interactive editing queue become empty, only then the processes on the batch queue will run.



The Description of the processes in the above diagram is as follows:

- **System Process** The Operating system itself has its own process to run and is termed as System Process.
- **Interactive Process** The Interactive Process is a process in which there should be the same kind of interaction (basically an online game).
- **Batch Processes** Batch processing is basically a technique in the Operating system that collects the programs and data together in the form of the **batch** before the **processing** starts.
- **Student Process** The system process always gets the highest priority while the student processes always get the lowest priority.

In an operating system, there are many processes, in order to obtain the result we cannot put all processes in a queue; thus this process is solved by Multilevel queue scheduling.

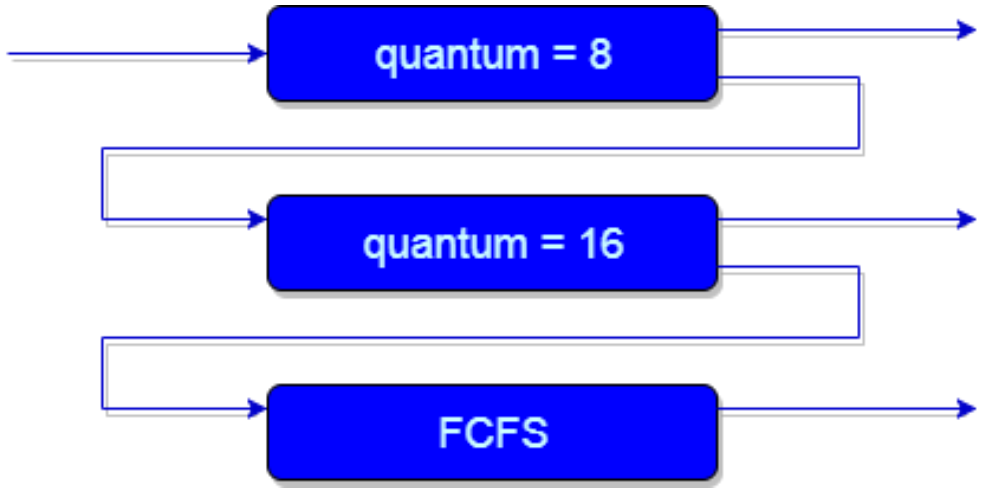
MULTILEVEL FEEDBACK QUEUE SCHEDULING

Multilevel feedback queue scheduling, however, allows a process to move between queues. The idea is to separate processes with different CPU-burst characteristics. If a process uses too much CPU time, it will be moved to a lower-priority queue. Similarly, a process that waits too long in a lower-priority queue may be moved to a higher-priority queue. This form of aging prevents starvation.

In general, a multilevel feedback queue scheduler is defined by the following parameters:

- The number of queues.
- The scheduling algorithm for each queue.
- The method used to determine when to upgrade a process to a higher-priority queue.
- The method used to determine when to demote a process to a lower-priority queue.
- The method used to determine which queue a process will enter when that process needs service.

The definition of a multilevel feedback queue scheduler makes it the most general CPU-scheduling algorithm. It can be configured to match a specific system under design. Unfortunately, it also requires some means of selecting values for all the parameters to define the best scheduler. Although a multilevel feedback queue is the **most general scheme**, it is also the **most complex**.



Explanation:

First of all, Suppose that queues 1 and 2 follow round robin with time quantum 8 and 16 respectively and queue 3 follows FCFS. One of the implementations of Multilevel Feedback Queue Scheduling is as follows:

1.If any process starts executing then firstly it enters queue 1.

2.In queue 1, the process executes for 8 unit and if it completes in these 8 units or it gives CPU for I/O operation in these 8 units unit than the priority of this process does not change, and if for some reasons it again comes in the ready queue than it again starts its execution in the Queue 1.

3.If a process that is in queue 1 does not complete in 8 units then its priority gets reduced and it gets shifted to queue 2.

4.Above points 2 and 3 are also true for processes in queue 2 but the time quantum is 16 units. Generally, if any process does not complete in a given time quantum then it gets shifted to the lower priority queue.

5.After that in the last queue, all processes are scheduled in an FCFS manner.

6.It is important to note that a process that is in a lower priority queue can only execute only when the higher priority queues are empty.

7.Any running process in the lower priority queue can be interrupted by a process arriving in the higher priority queue.

Advantages of MFQS

- This is a flexible Scheduling Algorithm
- This scheduling algorithm allows different processes to move between different queues.
- In this algorithm, A process that waits too long in a lower priority queue may be moved to a higher priority queue which helps in preventing starvation.

Disadvantages of MFQS

- This algorithm is too complex.
- As processes are moving around different queues which leads to the production of more CPU overheads.
- In order to select the best scheduler this algorithm requires some other means to select the values

CPU SCHEDULING ALGORITHM

- FCFS
- SJF
- PRIORITY
- RR
- MULTI LEVEL QUEUE
- MULTI LEVEL FEEDBACK QUEUE

Multiple Processor Scheduling

When multiple processors are available, then the scheduling gets more complicated, because now there is more than one CPU which must be kept busy and in effective use at all times.

Load sharing revolves around balancing the load between multiple processors.

Multi-processor systems may be heterogeneous, (different kinds of CPUs), or homogenous, (all the same kind of CPU).

1. Approaches to Multiple-Processor Scheduling

Asymmetric multiprocessing: One processor is the master, controlling all activities and running all kernel code, while the other runs only user code.

Symmetric multiprocessing (SMP): Each processor schedules its own jobs, either from a common ready queue or from separate ready queues for each processor.

2. Processor Affinity

Soft affinity occurs when the system attempts to keep processes on the same processor but makes no guarantees.

Linux and some other OSes support hard affinity, in which a process specifies that it is not to be moved between processors.

3. Load Balancing

Obviously an important goal in a multiprocessor system is to balance the load between processors, so that one processor won't be sitting idle while another is overloaded.

4. Multicore Processors

Traditional SMP required multiple CPU chips to run multiple kernel threads concurrently.

Recent trends are to put multiple CPUs (cores) onto a single chip, which appear to the system as multiple processors.

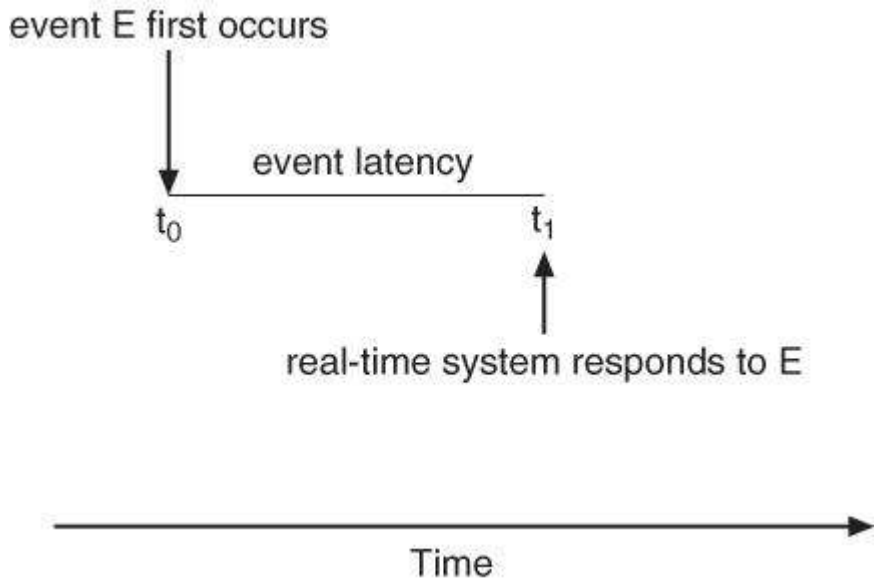
Real-Time CPU Scheduling

Real-time systems are those in which the time at which tasks complete is crucial to their performance

- ***Soft real-time systems*** have degraded performance if their timing needs cannot be met. Example: streaming video.
- ***Hard real-time systems*** have total failure if their timing needs cannot be met. Examples: Assembly line robotics, automobile air-bag deployment.

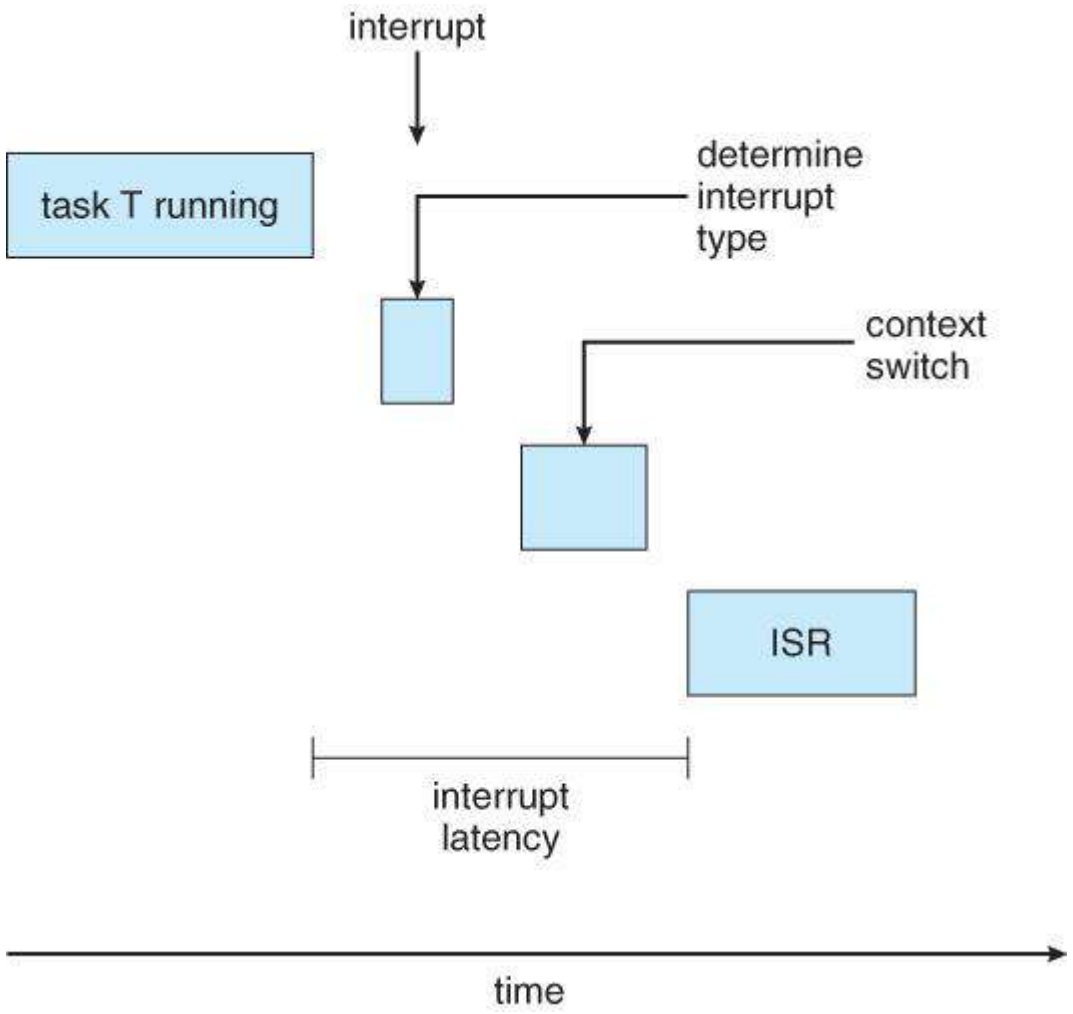
A. Minimizing Latency

- ***Event Latency*** is the time between the occurrence of a triggering event and the (completion of) the system's response to the event:

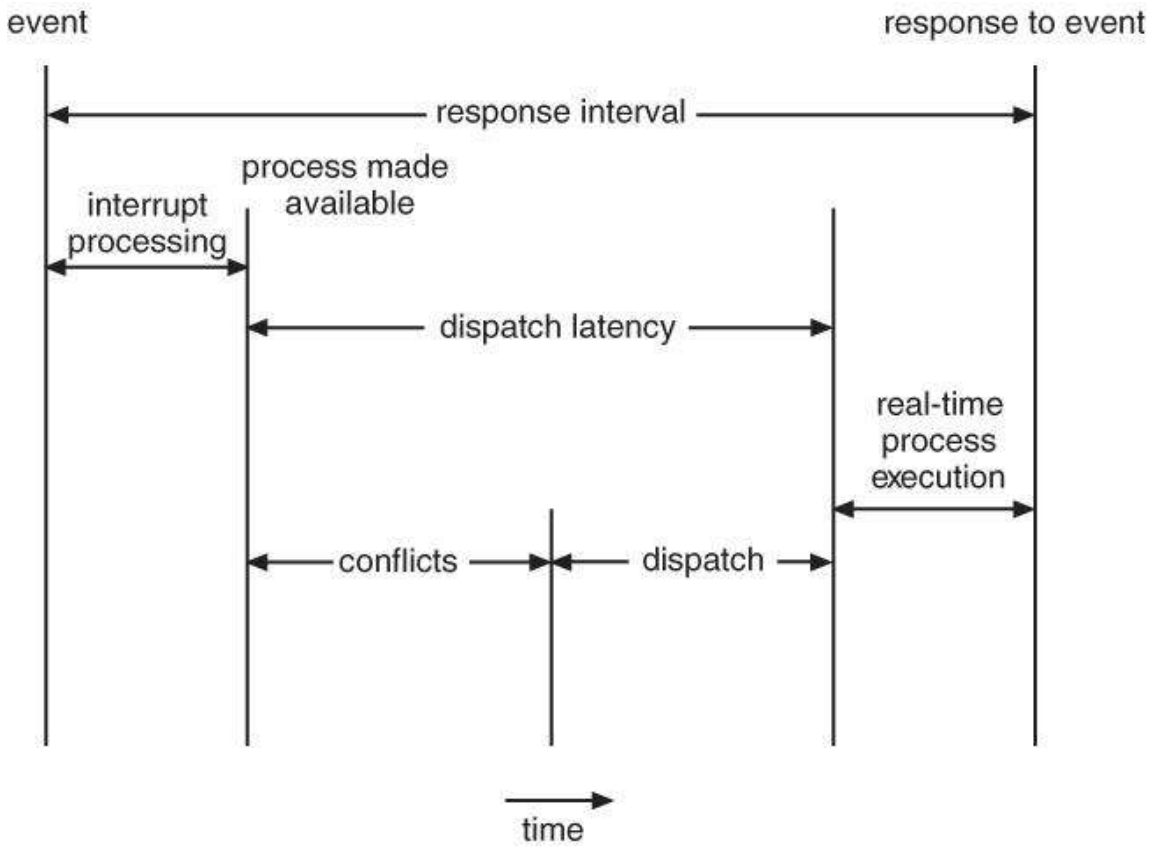


•In addition to the time it takes to actually process the event, there are two additional steps that must occur before the event handler (Interrupt Service Routine, ISR), can even start:

- Interrupt processing determines which interrupt(s) have occurred, and which interrupt handler routine to run. In the event of simultaneous interrupts, a priority scheme is needed to determine which ISR has the highest priority and gets to go next.
- Context switching involves removing a running process from the CPU, saving its state, and loading up the ISR so that it can run.



- The total dispatch latency (context switching) consists of two parts:
  - Removing the currently running process from the CPU, and freeing up any resources that are needed by the ISR. This step can be speeded up a lot by the use of pre-emptive kernels.
  - Loading the ISR up onto the CPU, (dispatching)

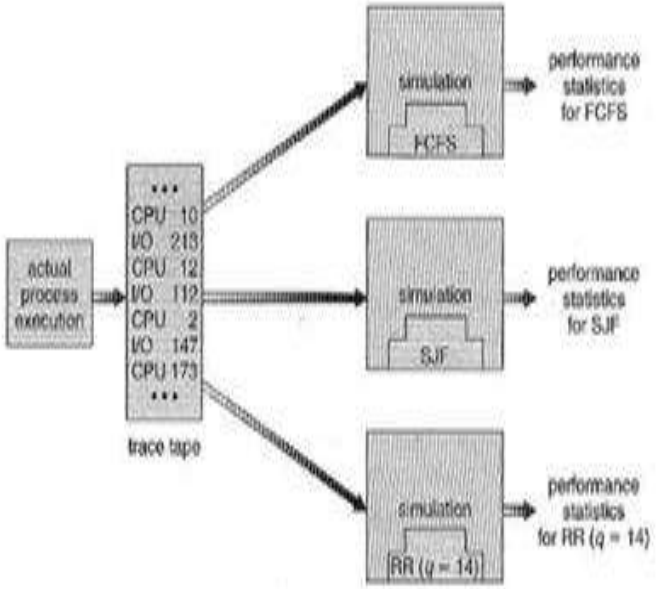


Algorithm Evaluation

Introduction: -Selection of an algorithm is difficult. The first problem is defining the criteria to be used in selecting an algorithm. Criteria are often defined in terms of CPU utilization, response time or through put. Criteria includes several measures such as-
a) Maximizing CPU utilization under the constraint that the maximum response time is 1 second.
b) Maximizing through put such that turnaround time is linearly proportional to total execution time.
Various evaluation methods that can be used-

Deterministic modeling- One major class of evaluation methods is **analytic evaluation**. Analytic evaluation uses the given algorithm and the system work load to produce a formula or number that evaluates the performance of the algorithm for that workload. One type of analytic evaluation is **deterministic modeling**. This method takes a particular pre determined work load and defines the performance of each algorithm for that work load. Deterministic modeling is simple and fast. It returns exact numbers allowing for comparison of algorithms. The main uses of deterministic modeling are in describing scheduling algorithms and providing examples.

Queuing models-On many systems, the processes that run vary from day to day so there is no static set of processes to use for deterministic modeling. The distribution of CPU and I/O bursts can be determined however. These distributions can then be measured and approximated. The result is a mathematical formula describing the probability of a particular CPU burst. The computer system is described as a network of servers. Each server has a queue of waiting processes. The CPU is a server with its ready queue as is the I/O system with its device queues. By knowing the arrival rates and service rates, utilization, average queue length, average wait time can be computed. This area of study is called **queuing network analysis**.



let n be the average queue length, let W be the average waiting time in the queue and let λ be the average arrival rate for new processes in the queue. If the system is in the steady state, then the number of processes leaving the queue must be equal to the number of processes that arrive. Thus,

$n = \lambda * W$

This equation known as Little's formula is useful because it is valid for any scheduling algorithm and arrival distribution. Queuing analysis can be useful in comparing scheduling algorithms.

Simulations:-Running simulations involves programming a model of the computer system. Running simulations involves programming a model of the computer system. Software data structures represent the major components of the system. The data to drive the simulation can be generated in several ways. The most common method uses a random number generator, which is programmed to generate processes, CPU burst times, arrivals, departures etc. The distributions can be defined mathematically (uniform, exponential, Poisson) or empirically. The results define the distribution of events in the real system; this distribution can then be used to drive the simulation. The frequency distribution indicates how many instances of each event occur; it does not indicate anything about the order of occurrence. To rectify this, we use trace tapes.

A trace tape is created by monitoring the real system and recording the sequence of actual events. This sequence is then used to drive the simulation. Trace tapes provide an excellent way to compare two algorithms on the same set of real inputs.
Simulations can be expensive, requiring hours of computer time. Trace tapes can require large amounts of storage space.

Implementation:- The only completely accurate way to evaluate a scheduling algorithm is to code it up, put it in the operating system and see how it works. This approach puts the actual algorithm in the real system for evaluation under real operating conditions.

The major difficulty with this approach is high cost. Another difficulty is that the environment in which the algorithm is used will change.

The most flexible scheduling algorithms are those that can be altered by the system managers or by the users so that they can be tuned for a specific application or set of applications. Another approach is to use API's that modify the priority of a process or thread.

IMPORTANT QUESTIONS – UNIT 1

SECTION A

1. Define Operating System
2. Name some operating system
3. List the features of Operating System
4. Define spooling
5. What is System call
6. What is Process
7. Difference between Process and Program
8. Define Scheduler
9. List the types of scheduler
10. What is IPC
11. Define Pre-emptive scheduling
12. Define non pre-emptive scheduling
13. What is PID
14. What is context switching

Section B

1. Explain any two types of operating system
2. Functions of OS
3. Operating System services
4. Components of OS
5. System Call
6. Explain Process state and PCB
7. Define Scheduler and its types
8. Scheduling Criteria
9. Explain the following CPU scheduling algorithm:
    1. FCFS
    2. SJF
    3. Priority
    4. Round Robin
10. Multi processor scheduling
11. Algorithm evaluation